Coding and Solution

Phase 5: Project Development Phase

1. Implementation Overview

The implementation connects a React frontend to a FastAPI backend. The user supplies trip preferences, the backend requests a structured itinerary from Groq, the AI service parses and validates the response, and the backend stores the result in Supabase. The frontend then renders the itinerary and provides actions for saved trips, map searches and printing.

2. Request Handling

The frontend sends trip details as JSON to the itinerary endpoint. FastAPI uses the request schema to validate the shape of the data before generation. A structured request reduces ambiguity and makes it easier to reproduce a test case. Validation should also consider empty fields, invalid dates and inappropriate numeric values.

3. Prompt and Model Call

The AI service builds a prompt from the starting city, destination, dates, budget, traveller count, interests and travel style. The configured Groq model is openai/gpt-oss-20b. The prompt asks for structured output so the frontend can display itinerary days and activities consistently. Model output should always be treated as untrusted content until parsing and validation complete.

4. Parsing and Validation

The service parses the model response as JSON and checks for required fields such as destination, summary, days, estimated total cost and travel tips. It also checks the nested day and activity structure. Markdown code fences or unexpected text can interfere with parsing, so response handling must either normalize the response safely or return a clear error.

5. Database Operations

After successful generation, the backend inserts trip details into the trips table and the generated itinerary into the itineraries table. Saved-trip endpoints list records, retrieve a selected record and delete the trip with its associated itinerary. A robust implementation should consider database failure after generation and avoid leaving records inconsistent.

6. Hotels and Restaurants Search

The place-search endpoints return Google Maps search URLs. Restaurant query terms can include a food preference such as vegetarian, non-vegetarian or Jain. Because the backend does not fetch individual place records, users must verify location, prices, availability, opening hours and dietary preparation in the external service.

7. Frontend Rendering and Print

The frontend renders the generated plan in a readable structure and offers Saved Trips actions. The print/save action relies on browser print functionality. Browser settings determine the final PDF layout, so users should preview the print output before saving or sharing it.

8. Configuration

The backend needs a Groq API key and Supabase connection configuration supplied through environment variables. The frontend needs the correct backend base URL for local or deployed operation. The values should differ by environment where needed, and private credentials must never be embedded in public frontend code.

9. Testing the Implementation

Test a successful generation request, malformed or missing inputs, model failure, database failure, saved-trip retrieval, deletion, map-search URLs and browser printing. Each test should capture an observed result and date. The source code alone does not establish that every path has been tested in the deployed environment.